You asked:

Has dicho

Tengo este problema.

[manjaro mnt]# manjaro-chroot . bash

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

 root /

 sudo su

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

 root /

 exit

exit

 root /

 exit

exit

[manjaro mnt]# manjaro-chroot .

sh-5.3# sudo su

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

 root /

 sddm

sddm: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.

so.6)

 root /

 sudo su

sudo: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_2.42' not found (required by /usr/lib/sudo/libsudo_util.

so.0)

 root /

 exit

exit

sh-5.3# exit

exit

Gemini Replied:

Diagnosing the Locale

I'm zeroing in on this Manjaro system's locale issue. The error message, specifically the `setlocale` warning tied to `es_AR.UTF-8` , is the primary focus. I believe I have the right focus.

Investigating the "GLIBC" Issue

I'm now zeroing in on the GLIBC errors, specifically the mismatch and dependency breakage. The presence of the Debian-style library path on Manjaro is a red flag. It points to a partial or failed upgrade, or perhaps an accidental mix-up of binaries from a different system. The `GLIBC_2.42` error definitely suggests something very bleeding edge, or at least that something went very wrong. I'm focusing on the root cause.

Addressing the Directory Issue

I'm now zeroing in on this exotic `/lib/x86_64-linux-gnu` path. My hypothesis is that this directory is not a standard Manjaro location. This suggests either a user-created misconfiguration or, more likely, an environment variable like `LD_LIBRARY_PATH` that's steering the system to an unexpected location. We need to clear this. We're going to give the user step by step instructions to address this.

Analyzing the Upgrade State

Me doy cuenta que `sudo` y `sddm` fallan por glibc. Es altamente probable que el sistema entero esté en un estado de "semi-actualización". Esto me da una idea de qué buscar. El problema raíz parece ser una inconsistencia de versiones en la librería central del sistema (glibc). Tengo que enfocarse en los pasos para resolver esto.

Gemini ha dicho

Esto es un problema crítico de **inconsistencia de versiones en la librería central (glibc)**. Básicamente, algunos programas de tu sistema (como `sudo` o `sddm`) se actualizaron y ahora buscan funciones de una versión de `glibc` que aún no tienes instalada, o tus rutas de librerías están mezcladas.

Lo más extraño es la ruta `/lib/x86_64-linux-gnu/`, que es típica de Ubuntu/Debian, no de Manjaro (donde todo suele estar en `/usr/lib`). Esto sugiere un error grave durante una actualización o una configuración de variables de entorno incorrecta.

Sigue estos pasos dentro del entorno de recuperación (Live USB) para intentar repararlo:

---

1. Entra al entorno chroot correctamente

Asegúrate de estar montando la partición correcta. Si ya estás en el prompt de `manjaro-chroot`, primero intentaremos limpiar las variables de entorno que podrían estar causando el conflicto de rutas.

Bash

```
# Entra al chroot limpiando el entorno
manjaro-chroot . /usr/bin/bash --login
```

2. Generar Locales (Para quitar los warnings)

Esto no arregla el error de `glibc`, pero limpiará tu terminal de mensajes molestos:

Bash

```
locale-gen
```

3. Sincronizar espejos y forzar actualización

El error de "GLIBC_2.42 not found" ocurre porque el sistema quedó a medias. Necesitamos que todo el sistema llegue a la última versión disponible.

1. **Actualiza la lista de servidores (mirrors):**

   Bash

   ```
   pacman-mirrors -f 5
   ```

2. **Fuerza la actualización de la base de datos y el sistema:**

Bash

```
pacman -Syyu
```

4. ¿Qué pasa si `pacman` no funciona?

Si al intentar el paso anterior `pacman` también te da el error de `glibc`, significa que las librerías base están rotas. En ese caso, debes usar el `pacman` del **Live USB** (el sistema que funciona) para instalar los paquetes en el **disco duro**:

Desde el terminal del Live USB (fuera del chroot, dale a `exit` si estás dentro):

Bash

```
# Reinstala glibc y pacman en el sistema dañado (asumiendo que está en /mnt)
sudo pacman -S glibc pacman --sysroot /mnt
```

¿Por qué ocurrió esto?

Este error suele aparecer por dos razones:

1. **Interrupción de actualización:** Se cortó la luz o se cerró la terminal mientras `pacman` actualizaba la librería `glibc`.
2. **Mezcla de repositorios:** Se intentaron instalar paquetes de una rama (como "Unstable") en un sistema de rama "Stable".

¿Lograste ejecutar el comando `pacman -Syu` dentro del chroot o te devolvió el mismo error de "version GLIBC not found"? Si me pasas el error exacto de pacman, puedo darte el comando específico para rescatar las librerías.

> **You asked:**

Has dicho

¿Cómo soluciono esto?

pacman_mirrors

> **Gemini Replied:**

Addressing the `pacman-mirrors` Issue

Okay, I'm focusing now on the user's Manjaro problem. It's about `pacman-mirrors`, likely connected to those earlier `chroot` and `glibc` issues. They want guidance on how to proceed. I am considering different approaches, weighing the best way forward. I am thinking this is important.

Gemini ha dicho

Para usar `pacman-mirrors` y refrescar tu lista de servidores (lo cual es vital si una actualización se quedó a medias), tienes un par de formas de hacerlo dependiendo de si el comando te da error de librerías o no.

Aquí tienes los comandos exactos y qué hace cada uno:

1. Si el comando funciona dentro del chroot

Si lograste entrar con `manjaro-chroot` y el comando responde, usa este para buscar los 5 servidores más rápidos y actualizados:

Bash

```
pacman-mirrors -f 5
```

2. Si quieres servidores de un país específico (recomendado)

A veces los servidores globales fallan. Puedes forzar que busque los de tu región (por ejemplo, Argentina y Chile) para asegurar mejor conexión:

Bash

```
pacman-mirrors --country Argentina,Chile && pacman-mirrors -f 5
```

3. Si `pacman-mirrors` NO funciona (Error de GLIBC)

Si al intentar ejecutar el comando recibes el mismo error de `libc.so.6`, significa que no puedes usar las herramientas del sistema dañado. Tienes que ejecutarlo **desde el sistema del Live USB** (fuera del chroot):

1. Sal del chroot si estás dentro: `exit`.

2. Ejecuta en la terminal del Live USB:

   Bash

   ```
   sudo pacman-mirrors -f 5
   ```

   Esto actualizará la lista de espejos del Live USB, que luego usaremos para reparar el disco duro.

EL PASO CRUCIAL: Sincronizar después de los mirrors

Cambiar los mirrors no arregla el sistema por sí solo; solo le dice a `pacman` de dónde bajar los archivos nuevos. Inmediatamente después de correr `pacman-mirrors`, debes intentar actualizar:

Si estás dentro del chroot:

Bash

```
pacman -Syyu
```

Si estás fuera del chroot (porque el sistema está roto): Usa el pacman del Live USB para "inyectar" la actualización en tu disco (asumiendo que tu disco está montado en `/mnt`):

Bash

```
sudo pacman -Syyu --sysroot /mnt
```

¿Te lanzó algún error específico al intentar correr `pacman-mirrors` o simplemente no sabías qué parámetros poner? Si te da el error de `GLIBC_2.42`, definitivamente tenemos que usar el método `--sysroot`.

> **You asked:**

Has dicho

Quiero que pacman-mirrors funcione dentro de root, ¿Se puede hacer algo si fuera funciona?

> **Gemini Replied:**

Addressing the Glitch

I'm zeroing in on the `glibc` mismatch in the `chroot` environment. The previous response confirmed this as the key issue with `pacman-mirrors` . The user now wants...

Isolating the Root Cause

I've clarified the root problem: a `glibc` mismatch within the `chroot` environment is preventing `pacman-mirrors` from functioning. The host system's compatibility is confirmed. My next move is to outline the differing states, highlighting the broken libraries of the guest. I need to explain the disconnect, then provide solutions to address the "guest" system's package state so tools can operate correctly.

Gemini ha dicho

Sí, totalmente. Si `pacman-mirrors` funciona afuera (en el Live USB), podemos usar ese éxito para "inyectarle" la solución al sistema roto. El problema es que dentro del chroot el programa intenta usar una librería (`glibc`) que está dañada o es incompatible, por eso falla.

Aquí tienes el plan de rescate usando el sistema que sí funciona como puente:

1. Actualiza los mirrors desde AFUERA

Sal del chroot (escribe `exit` hasta que veas el prompt del Live USB). Luego ejecuta el comando en el sistema que sí funciona:

Bash

```
sudo pacman-mirrors -f 5
```

Esto genera un archivo de servidores actualizado en `/etc/pacman.d/mirrorlist` **del Live USB**.

2. Pásale la lista al sistema roto (El truco del "Copy-Paste")

Como tu sistema en el disco duro está montado en `/mnt` , simplemente copia el archivo de servidores que acabas de generar hacia el disco:

Bash

```
sudo cp /etc/pacman.d/mirrorlist /mnt/etc/pacman.d/mirrorlist
```

Con esto, aunque `pacman-mirrors` no abra dentro del chroot, el archivo que necesita ya estará ahí y estará actualizado.

3. El paso maestro: Reparar con "Inyección"

Ahora, en lugar de entrar al chroot y esperar que las herramientas rotas funcionen, usaremos el `pacman` sano del Live USB para arreglar el `glibc` del disco duro.

Ejecuta esto desde la terminal del Live USB:

Bash

```
sudo pacman -Syu glibc lib32-glibc pacman --sysroot /mnt
```

¿Qué hace este comando?

- **`-Syu`**: Sincroniza y actualiza.
- **`glibc lib32-glibc pacman`**: Reinstala los componentes base que están dando el error de "version not found".
- **`--sysroot /mnt`**: Es la clave. Le dice a `pacman`: *"Usa tu motor y tu conexión, pero instala todo dentro de /mnt como si fuera el sistema principal"*.

---

4. Vuelve a intentar el chroot

Una vez que termine la instalación del paso anterior, las librerías básicas deberían estar sanas. Ahora intenta entrar de nuevo:

Bash

```
manjaro-chroot /mnt /usr/bin/bash --login
```

Si entras y ya no ves los errores de `GLIBC`, ¡victoria! Ahora podrías correr `pacman-mirrors` o cualquier otro comando normalmente.

¿El comando del paso 3 terminó sin errores o te dijo algo sobre "conflicting files" (conflictos de archivos)? Si hay conflictos, avísame, porque habrá que usar el parámetro `--overwrite`.

> **You asked:**

Has dicho

No funciona la inyección.

> **Gemini Replied:**

Analyzing the Glitch

I'm focusing on the user's "No funciona la inyección" feedback. My last suggestion was the Live USB `glibc` repair command. The user's response likely indicates a failure of that approach, which means I should start evaluating other potential root causes. I need to consider various factors beyond a simple `glibc` problem.

Investigating the System

I'm now diving deeper into the root cause, focusing on the system's architecture. The `/lib/x86_64-linux-gnu/` path in the logs is a glaring red flag. I believe the library path points to a serious mismatch or installation issue. This path is Debian/Ubuntu based not Arch/Manjaro. I need to figure out why a Debian path is even showing up on an Arch-based system. I am moving towards finding out why the Live USB solution failed. I'll focus on error messages, correct mount points and signature issues as my next steps.

Gemini ha dicho

Si la "inyección" falló, es probable que sea por una de estas tres razones: un error de firmas (PGP Keys), archivos que ya existen en el sistema (conflicto de archivos) o simplemente que la ruta `/mnt` no es donde está tu sistema.

Para ayudarte a tiro fijo, **necesito que me copies el error exacto que te dio el comando anterior**.

Mientras tanto, intentemos el "Plan B", que es más agresivo y suele saltarse los bloqueos de librerías:

1. Asegúrate de que el disco esté bien montado

A veces el Live USB pierde el montaje. Verifica con:

Bash

```
ls /mnt/etc/hostname
```

Si no devuelve el nombre de tu PC, el disco no está en `/mnt` y nada de lo que hagamos funcionará.

2. Intenta la reparación con `-r` (el método clásico)

Si `--sysroot` falló, probemos con el parámetro `--root`. Ejecuta esto desde el terminal del Live USB (fuera del chroot):

Bash

```
sudo pacman -Syu glibc lib32-glibc pacman --root /mnt --cachedir /mnt/var/cache/pacman/pkg --over
```

¿Por qué este comando es distinto?

- **`--root /mnt`** : Fuerza a pacman a tratar `/mnt` como la base del sistema.
- **`--cachedir`** : Evita problemas de espacio usando la caché del sistema en el disco.
- **`--overwrite "*"`** : **Este es el más importante.** Si tu sistema tiene archivos de `glibc` a medio instalar o corruptos, pacman se detendrá por "conflicto de archivos". Esto obliga a pacman a escribir encima de lo que esté roto.

3. Si el problema son las "Llaves" (PGP Keys)

Si el error dice algo como "invalid or corrupted package (PGP signature)", corre esto antes de volver a intentar el paso 2:

Bash

```
sudo pacman -Sy archlinux-keyring manjaro-keyring --root /mnt
```

Un detalle vital: La ruta extraña

En tu primer mensaje apareció esto:
`sddm: /lib/x86_64-linux-gnu/libc.so.6: version ... not found`

Esa ruta **no pertenece a Manjaro**, pertenece a sistemas como Debian o Ubuntu. Si después de reinstalar `glibc` el error persiste, es posible que tengas una variable de entorno llamada `LD_LIBRARY_PATH` que está apuntando a una carpeta que no debería existir.

¿Qué error te sale ahora al intentar el paso 2? Si me pasas las últimas 5 líneas del error, te digo exactamente qué "llave" o qué "archivo" está bloqueando la reparación.

> **You asked:**

Has dicho

Lo que quiero es copiarle el pacman-mirrors que tiene el sistema fuera de chroot adentro de chroot.

> ✦ **Gemini Replied:**

Transferring pacman-mirrors Config

I'm focusing on transferring the `pacman-mirrors` configuration and binary from the Live USB to the chroot environment. The version in the chroot is problematic due to `glibc` issues. I plan on copying the files directly from the Live USB to the chroot to resolve this.

Analyzing Glibc Compatibility

I've been analyzing the `glibc` compatibility issue with `pacman-mirrors`. The core problem is the differing `glibc` versions between the Live USB and the chroot environment. Copying the binary, as requested, *might* offer a temporary workaround. I'm focusing on providing the user with explicit steps to do so, while also warning them about the possible side effects and limitations. The target paths are `/usr/bin/pacman-mirrors` for the binary, and `/etc/pacman-mirrors.conf` and `/etc/pacman.d/mirrorlist` for configuration files. The "Infection" method is copying the binary from the Live system to `/mnt/usr/bin/` on the broken system.

Evaluating Dependency Challenges

I'm now focusing on a more nuanced approach. Copying just the `pacman-mirrors` script is unlikely to work due to its Python dependencies, which also rely on the problematic `glibc`. I'm going to explain that if `libc.so.6` is corrupted, nothing in the chroot environment will execute. I'll provide the `cp` commands requested, but I'll strongly emphasize that they are a temporary "Band-Aid" and won't resolve the core `glibc` issue. I am also considering suggesting copying only the final mirrorlist file, as a safer alternative.

Gemini ha dicho

Entiendo perfectamente tu lógica: si el martillo de adentro está roto, metamos el martillo de afuera que sí funciona.

Sin embargo, hay un detalle técnico: `pacman-mirrors` en Manjaro no es un "archivo único" (binario estático), sino un script de **Python**. Si lo copias así nada más, seguirá fallando porque al intentar ejecutarse dentro de la "habitación" (chroot) rota, llamará al Python de adentro, el cual llamará a la `glibc` de adentro... y volverás al error del principio.

Aun así, si quieres intentarlo para ver si logras saltarte el error, aquí tienes cómo "clonar" el estado de `pacman-mirrors` de afuera hacia adentro:

1. Copiar el ejecutable y la configuración

Ejecuta esto desde el Live USB (fuera del chroot):

Bash

```
# Copia el script principal
sudo cp /usr/bin/pacman-mirrors /mnt/usr/bin/pacman-mirrors

# Copia la configuración de los mirrors
sudo cp /etc/pacman-mirrors.conf /mnt/etc/pacman-mirrors.conf

# Copia la lista de servidores ya generada (esto es lo más importante)
sudo cp /etc/pacman.d/mirrorlist /mnt/etc/pacman.d/mirrorlist
```

2. El problema de las "piezas" (Python)

`pacman-mirrors` necesita sus librerías de Python. Si la versión de Python en el Live USB es la misma que la del sistema instalado, podrías intentar copiar la carpeta de la librería (ajusta `python3.12` a la versión que veas en `/usr/lib/`):

Bash

```
sudo cp -r /usr/lib/python3.12/site-packages/pacman_mirrors /mnt/usr/lib/python3.12/site-packages
```

⚠️ Por qué esto probablemente no te dé el resultado que buscas

Tu error dice: `version GLIBC_2.42 not found`. Esto es como si hubieras cambiado el motor de un auto (`glibc`) pero dejaste los tornillos viejos. Nada que intentes ejecutar **dentro** del chroot va a funcionar mientras `libc.so.6` esté corrupta o incompleta.

La solución real a lo que quieres (usar el pacman-mirrors de afuera para arreglar adentro) es esta:

1. **Genera los mirrors afuera:** `sudo pacman-mirrors -f 5`
2. **Copia la lista adentro:** `sudo cp /etc/pacman.d/mirrorlist /mnt/etc/pacman.d/mirrorlist`
3. **Usa el "instalador" de afuera para arreglar los cimientos de adentro:**

   Bash

   ```
   sudo pacman -S glibc lib32-glibc manjaro-system --root /mnt --overwrite "*"
   ```

Este tercer paso hace exactamente lo que pides: usa el motor de `pacman` que funciona (el de afuera) para sobreescribir las librerías rotas de adentro. Una vez que eso termine, podrás entrar al chroot y todo funcionará mágicamente.

¿Intentaste copiar el archivo `mirrorlist` manualmente como puse en el paso 1? Eso suele ser suficiente para que `pacman` (usado con `--root /mnt`) sepa a dónde ir a buscar los archivos sanos.

> **You asked:**

Has dicho

[manjaro compilar]# # Copia el script principal

sudo cp /usr/bin/pacman-mirrors /mnt/usr/bin/pacman-mirrors

Copia la configuración de los mirrors

sudo cp /etc/pacman-mirrors.conf /mnt/etc/pacman-mirrors.conf

Copia la lista de servidores ya generada (esto es lo más importante)

sudo cp /etc/pacman.d/mirrorlist /mnt/etc/pacman.d/mirrorlist

[manjaro compilar]# manjaro-chroot /mnt bash

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

 root /

 pacman-mirrors --country Argentina,Chile && pacman-mirrors -f 5

Traceback (most recent call last):

 File "/usr/bin/pacman-mirrors", line 5, in <module>

   from pacman_mirrors.pacman_mirrors import start

ModuleNotFoundError: No module named 'pacman_mirrors'

 root /

 exit

exit

[manjaro compilar]# sudo cp -r /usr/lib/python3.12/site-packages/pacman_mirrors /mnt/usr/lib/python3.12/site-p

ackages/

cp: cannot stat '/usr/lib/python3.12/site-packages/pacman_mirrors': No such file or directory

[manjaro compilar]# ls -lh /mnt/usr/lib/python3.1/site-packages/

python3.13/ python3.14/

[manjaro compilar]# ls -lh /mnt/usr/lib/python3.14^Cite-packages/

[manjaro compilar]# sudo cp -r /usr/lib/python3.12/site-packages/pacman_mirrors /mnt/usr/lib/python3.1/site-pa

ckages/

python3.13/ python3.14/

[manjaro compilar]# sudo cp -r /usr/lib/python3.13/site-packages/pacman_mirrors /mnt/usr/lib/python3.14/site-p

ackages/

python3.13/

[manjaro compilar]# sudo cp -r /usr/lib/python3.13/site-packages/pacman_mirrors /mnt/usr/lib/python3.14/site-p

ackages/

[manjaro compilar]# manjaro-chroot /mnt bash

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

 root /

 pacman-mirrors --country Argentina,Chile && pacman-mirrors -f 5

Traceback (most recent call last):

 File "/usr/bin/pacman-mirrors", line 5, in <module>

   from pacman_mirrors.pacman_mirrors import start

ModuleNotFoundError: No module named 'pacman_mirrors'

 root /

 python --version

Python 3.13.5

 root /

 exit

exit

[manjaro compilar]# sudo cp -r /usr/lib/python3.13/site-packages/pacman_mirrors /mnt/usr/lib/python3.1/site-pa

ckages/

python3.13/ python3.14/

[manjaro compilar]# sudo cp -r /usr/lib/python3.13/site-packages/pacman_mirrors /mnt/usr/lib/python3.13/site-p

ackages/

[manjaro compilar]# manjaro-chroot /mnt bash

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

 root /

 python --version

Python 3.13.5

 root /

 pacman-mirrors --country Argentina,Chile && pacman-mirrors -f 5

Traceback (most recent call last):

 File "/usr/bin/pacman-mirrors", line 5, in <module>

   from pacman_mirrors.pacman_mirrors import start

ModuleNotFoundError: No module named 'pacman_mirrors'

 root /

 exit

exit

Gemini Replied:

Troubleshooting Broken Install

I'm currently focused on resolving the `glibc` mismatch encountered while trying to use `pacman` from the live environment to fix the broken Manjaro install. I've been experimenting with chrooting into the system to see if I can rectify the glibc issue directly. Exploring possible solutions, like downgrading glibc or updating packages, looks like the next best step.

Investigating Pacman-Mirrors Failure

I'm now zeroing in on why `pacman-mirrors` is failing within the chroot. Despite copying the script and its dependencies from the Live USB, I'm getting a `ModuleNotFoundError`. It seems a path mismatch or version conflict is likely. I've confirmed Python 3.13 exists in both environments, but the complexity of `pacman-mirrors` and its dependencies suggests other hidden dependencies may be missing or mismatched in the chroot environment, and manual copying has proved fruitless.

Pinpointing Module Search Failure

I'm now focusing on pinpointing *precisely* why the `pacman_mirrors` module isn't being found despite the copy. The user's prompt strongly indicates the copy was successful. My first thought is a version mismatch, or potentially a script-level path issue. I need to verify the exact module location on the Live USB, and explain Python search paths for modules. The provided logs seem to confirm the copy went through, so the focus shifts to how Python inside the chroot actually searches.

Clarifying Module Location

I'm now zeroing in on the file paths. The prompt shows a folder, which could be relevant. I'm thinking the user needs explicit instructions: First, they'll check the Live USB's module path. Then, they'll verify the chroot's path. The user's `/mnt/usr/lib/python3.14/` is a potential red flag, indicating a 3.14 version mismatch. We'll give commands to get both paths, and then they can attempt the correct copy. If that fails, I'll explain `pacman-mirrors` relies on more than just the module folder. However, I believe that we can make this work if we just focus on `/etc/pacman`.

Gemini ha dicho

Te entiendo, quieres que la herramienta nativa funcione dentro para no depender de "trucos" externos. El problema es que copiar carpetas de Python a mano es como intentar trasplantar un órgano: si no conectas todas las arterias (dependencias), el cuerpo lo rechaza.

El error `ModuleNotFoundError` ocurre porque el Python del sistema dentro del chroot no está encontrando la carpeta que copiaste o le faltan otras piezas.

Aquí tienes los pasos exactos para intentar que `pacman-mirrors` reviva adentro:

1. Localiza exactamente dónde está el módulo afuera

En el Live USB (fuera del chroot), averigua dónde vive realmente el paquete:

Bash

```
python -c "import pacman_mirrors; print(pacman_mirrors.__path__)"
```

Esto te dirá la ruta exacta (probablemente `/usr/lib/python3.13/site-packages/pacman_mirrors`).

2. Verifica la versión de Python adentro

Ya vimos que tienes la **3.13.5** adentro. Asegúrate de que la ruta de destino coincida exactamente.

Ejecuta esto **afuera** para copiarlo a la carpeta correcta de Python 3.13:

Bash

```
sudo cp -r /usr/lib/python3.13/site-packages/pacman_mirrors /mnt/usr/lib/python3.13/site-packages
```

3. El "Secreto": Las dependencias de pacman-mirrors

`pacman-mirrors` no es solo esa carpeta. También necesita otros módulos de Python para funcionar (como `requests`, `gi`, `yaml`, etc.). Si esos módulos están corruptos en tu disco duro por el problema de `glibc`, el script seguirá fallando.

Para ver qué le falta, entra al chroot e intenta importarlo manualmente:

Bash

```
manjaro-chroot /mnt bash
python3 -c "from pacman_mirrors.pacman_mirrors import start"
```

Si te da error de otro módulo (ej. `ModuleNotFoundError: No module named 'yaml'`), tendrías que copiar también esa carpeta.

La alternativa: El "Bypass" (Si lo anterior falla)

Si tu objetivo final es que el comando `pacman -Syu` funcione adentro y necesitas los mirrors actualizados, puedes "engañar" al sistema.

En lugar de arreglar el script, dale el resultado del script:

1. **Afuera:** `sudo pacman-mirrors -f 5` (Esto crea el archivo perfecto).
2. **Copia el resultado:** `sudo cp /etc/pacman.d/mirrorlist /mnt/etc/pacman.d/mirrorlist`
3. **Adentro:** Ahora simplemente corre `pacman -Syyu`. No necesitas que `pacman-mirrors` funcione porque ya le diste la lista de servidores lista para usar.

Un aviso importante sobre GLIBC

Incluso si logras que `pacman-mirrors` arranque, lo más probable es que cuando intente escribir el archivo o hacer una conexión de red, choque contra el error de **`GLIBC_2.42 not found`**.

Tu sistema está en un estado "híbrido": tienes binarios (programas) que esperan una versión de sistema que aún no terminaste de instalar.

¿Qué te sale si intentas correr `pacman -Syyu` dentro del chroot ahora que ya tienes el mirrorlist copiado? Si `pacman` funciona, el problema está casi resuelto. Si `pacman` te da error de `GLIBC`, tenemos que arreglar la librería primero desde afuera.

> **You asked:**

Has dicho

[manjaro compilar]# sudo cp -r /usr/lib/python3.13/site-packages/pacman_mirrors /mnt/usr/lib/python3.13/site-p

ackages/

[manjaro compilar]# manjaro-chroot /mnt bash

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

 root /

 python -c "import pacman_mirrors; print(pacman_mirrors.__path__)"

Traceback (most recent call last):

 File "<string>", line 1, in <module>

   import pacman_mirrors; print(pacman_mirrors.__path__)

   ^^^^^^^^^^^^^^^^^^^^^^

ModuleNotFoundError: No module named 'pacman_mirrors'

 root /

 pacman-mirrors --country Argentina,Chile && pacman-mirrors -f 5

Traceback (most recent call last):

 File "/usr/bin/pacman-mirrors", line 5, in <module>

   from pacman_mirrors.pacman_mirrors import start

ModuleNotFoundError: No module named 'pacman_mirrors'

 root /

 exit

exit

[manjaro compilar]# manjaro-chroot /mnt bash

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

 root /

 python3 -c "from pacman_mirrors.pacman_mirrors import start"

Traceback (most recent call last):

 File "<string>", line 1, in <module>

  from pacman_mirrors.pacman_mirrors import start

ModuleNotFoundError: No module named 'pacman_mirrors'

 root /

 exit

exit

> **Gemini Replied:**

Gemini ha dicho

Parece que Python dentro del chroot se está resistiendo a ver esa carpeta. Esto suele pasar por un tema de **permisos** o porque el **PATH** de búsqueda de Python en el sistema roto es distinto al del Live USB.

Hagamos un último intento para forzar a Python a que reconozca el módulo, asegurándonos de que los permisos sean los correctos (a veces al copiar desde afuera quedan como `root` y el usuario del chroot no los lee bien).

1. Corregir permisos y verificar ruta (Desde AFUERA)

Ejecuta esto en el Live USB:

Bash

```
# Asegurarnos de que la carpeta existe y tiene los permisos correctos
sudo chmod -R 755 /mnt/usr/lib/python3.13/site-packages/pacman_mirrors
sudo chown -R root:root /mnt/usr/lib/python3.13/site-packages/pacman_mirrors

# Verificar si hay archivos dentro realmente
ls -l /mnt/usr/lib/python3.13/site-packages/pacman_mirrors
```

2. Forzar el reconocimiento (Desde ADENTRO)

Entra al chroot y dile a Python dónde buscar explícitamente usando la variable `PYTHONPATH`:

Bash

```
manjaro-chroot /mnt bash
export PYTHONPATH=$PYTHONPATH:/usr/lib/python3.13/site-packages/
pacman-mirrors --country Argentina,Chile && pacman-mirrors -f 5
```

Si eso sigue fallando... la realidad del problema

Si después de esto sigue diciendo `ModuleNotFoundError`, el problema no es solo la carpeta de `pacman_mirrors`. Es probable que el paquete **`python-setuptools`** o las librerías base de Python en tu sistema instalado estén rotas debido al error de `GLIBC_2.42` que vimos al principio.

Recuerda lo más importante: No necesitas que el comando `pacman-mirrors` funcione para arreglar tu sistema. Si lo que quieres es actualizar:

1. **Copia el archivo ya hecho (Afuera):**
   `sudo pacman-mirrors -f 5 && sudo cp /etc/pacman.d/mirrorlist /mnt/etc/pacman.d/mirrorlist`
2. **Repara el sistema (Afuera):**
   `sudo pacman -Syu glibc manjaro-system --root /mnt --overwrite "*"`

¿Qué te devolvió el comando `ls -l` del paso 1? Si la carpeta aparece vacía o no existe, es que el comando `cp` anterior falló silenciosamente. Si la carpeta tiene archivos, el problema es puramente de Python y su entorno.

> **You asked:**

Has dicho

[manjaro compilar]# # Asegurarnos de que la carpeta existe y tiene los permisos correctos

```
sudo chmod -R 755 /mnt/usr/lib/python3.13/site-packages/pacman_mirrors

sudo chown -R root:root /mnt/usr/lib/python3.13/site-packages/pacman_mirrors


# Verificar si hay archivos dentro realmente

ls -l /mnt/usr/lib/python3.13/site-packages/pacman_mirrors

total 12

drwxr-xr-x 1 root root   88 mar 17 04:04 api

drwxr-xr-x 1 root root  134 mar 17 04:04 builder

drwxr-xr-x 1 root root   76 mar 17 04:04 config

drwxr-xr-x 1 root root   98 mar 17 04:04 constants

drwxr-xr-x 1 root root   96 mar 17 04:04 dialogs

drwxr-xr-x 1 root root  528 mar 17 04:04 functions

-rwxr-xr-x 1 root root   20 mar 17 04:05 __init__.py

drwxr-xr-x 1 root root   62 mar 17 04:04 mirrors

-rwxr-xr-x 1 root root 5185 mar 17 04:05 pacman_mirrors.py

drwxr-xr-x 1 root root  240 mar 17 04:04 __pycache__

drwxr-xr-x 1 root root  106 mar 17 04:04 translation

[manjaro compilar]# manjaro-chroot /mnt bash

export PYTHONPATH=$PYTHONPATH:/usr/lib/python3.13/site-packages/

pacman-mirrors --country Argentina,Chile && pacman-mirrors -f 5

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

 root / 

 export PYTHONPATH=$PYTHONPATH:/usr/lib/python3.13/site-packages/

pacman-mirrors --country Argentina,Chile && pacman-mirrors -f

Traceback (most recent call last):

 File "/usr/bin/pacman-mirrors", line 5, in <module>

   from pacman_mirrors.pacman_mirrors import start
```

File "/usr/lib/python3.13/site-packages/pacman_mirrors/pacman_mirrors.py", line 30, in <module>

from pacman_mirrors.functions import util

File "/usr/lib/python3.13/site-packages/pacman_mirrors/functions/util.py", line 29, in <module>

from pacman_mirrors.functions import httpFn

File "/usr/lib/python3.13/site-packages/pacman_mirrors/functions/httpFn.py", line 32, in <module>

import requests

ModuleNotFoundError: No module named 'requests'

 root /

 exit

exit

Gemini Replied:

Gemini ha dicho

¡Avanzamos! Ya lograste que Python encuentre la carpeta, pero ahora estamos en el juego del "gato y el ratón": `pacman-mirrors` necesita el módulo **`requests`**, y después de ese probablemente te pida `yaml`, y así sucesivamente.

Como tu sistema tiene el problema de `GLIBC`, no puedes usar `pip` ni `pacman` dentro del chroot para instalar lo que falta. La solución es seguir tu lógica de "copiar lo de afuera" pero de forma masiva para que no te falte ninguna pieza.

La solución definitiva para los módulos

En lugar de copiar solo `pacman_mirrors`, copia **todo** el contenido de `site-packages` del Live USB al sistema roto. Esto asegurará que `requests` y todas sus dependencias estén presentes.

Ejecuta esto desde el **Live USB** (fuera del chroot):

Bash

```
# Copiamos TODO el contenido de las librerías de Python del Live USB al disco
sudo cp -rn /usr/lib/python3.13/site-packages/* /mnt/usr/lib/python3.13/site-packages/
```

(Uso `-n` para "no sobreescribir", así solo copiamos lo que falta y no rompemos lo que ya esté ahí).

¿Qué pasa si `cp` falla o sigue pidiendo cosas?

Si después de copiar todo sigue fallando, es porque el problema de raíz no es que falten archivos, sino que los archivos que están ahí **no pueden ejecutarse** porque la `GLIBC` del disco es incompatible con el Python del Live USB.

Si quieres arreglar el sistema YA (el método infalible):

1. **Genera la lista afuera:**

   Bash

   ```
   sudo pacman-mirrors -f 5
   ```

2. **Pásala adentro:**

   Bash

   ```
   sudo cp /etc/pacman.d/mirrorlist /mnt/etc/pacman.d/mirrorlist
   ```

3. **Repara las librerías base desde afuera:** Este comando usa el `pacman` sano para sobreescribir el desastre de adentro:

   Bash

   ```
   sudo pacman -Syu glibc lib32-glibc manjaro-system --root /mnt --overwrite "*"
   ```

Una vez que ese comando termine, podrás entrar por `manjaro-chroot` y **todo**, incluyendo `pacman-mirrors`, funcionará porque habrás reparado los cimientos del sistema.

¿Prefieres seguir intentando copiar módulos de Python uno a uno o probamos la reparación de la glibc desde afuera? Si eliges la glibc, estarás a un paso de reiniciar y tener el sistema funcional.

> **You asked:**

Has dicho

 pacman-mirrors --country Argentina,Chile && pacman-mirrors -f

::AVISO Opción 'c/--country' : 'c/--country' : 'país desconocido'

::INFORMACION Los países disponibles son

Australia, Austria, Bangladesh, Belarus, Belgium, Brazil, Bulgaria, Canada, Chile, China, Colombia, Costa_Rica

, Czech_Republic, Denmark, France, Georgia, Germany, Global, Greece, Hungary, Indonesia, Iran, Italy, Japan, K

enya, Mauritius, Netherlands, New_Zealand, Poland, Portugal, Russia, Serbia, Singapore, Slovenia, South_Africa

, Spain, Sweden, Switzerland, Taiwan, Thailand, Turkey, Ukraine, United_Kingdom, United_States, Uruguay, Uzbek

istan, Vietnam

 root /

 pacman-mirrors --country Uruguay && pacman-mirrors -f

::INFORMACION Bajar réplicas desde Manjaro

::INFORMACION => Mirror pool: https://repo.manjaro.org/mirrors.json

::INFORMACION => Mirror status: https://repo.manjaro.org/status.json

::INFORMACION Lista de réplicas generada por el usuario

::----------

::INFORMACION Guardado el archivo de réplicas personalizado: /var/lib/pacman-mirrors/custom-mirrors.json

::INFORMACION Usando archivo de mirrors por defecto

::INFORMACION Consultando mirrors - Esto puede tomar un tiempo

0.536 Uruguay : https://manjaro.repo.cure.edu.uy/

::INFORMACION Escribiendo lista de mirrors

::Uruguay : https://manjaro.repo.cure.edu.uy/stable

::INFORMACION La lista de réplicas generada y guardada en: /etc/pacman.d/mirrorlist

::INFORMACION Bajar réplicas desde Manjaro

::INFORMACION => Mirror pool: https://repo.manjaro.org/mirrors.json

::INFORMACION => Mirror status: https://repo.manjaro.org/status.json

::INFORMACION Usando archivo de mirrors personalizado

::INFORMACION Consultando mirrors - Esto puede tomar un tiempo

0.407 Uruguay : https://manjaro.repo.cure.edu.uy/

::INFORMACION Escribiendo lista de mirrors

::Uruguay : https://manjaro.repo.cure.edu.uy/stable/$repo/$arch

::INFORMACION La lista de réplicas generada y guardada en: /etc/pacman.d/mirrorlist

 root /

Gemini Replied:

Gemini ha dicho

¡Excelente trabajo! Has logrado que `pacman-mirrors` funcione perfectamente dentro del chroot. Esto significa que el entorno de Python ya tiene las librerías básicas necesarias y el sistema ya sabe de dónde descargar los paquetes sanos (en este caso, desde el servidor de Uruguay).

Ahora que la lista de mirrors está actualizada y guardada en `/etc/pacman.d/mirrorlist`, es el momento de **reparar el sistema**.

Sigue este orden exacto para resolver el error de la `GLIBC` y las inconsistencias que vimos al principio:

1. Sincronizar y Actualizar la base de datos

Dentro del chroot donde estás ahora, ejecuta:

Bash

```
pacman -Syy
```

(El doble `y` es para forzar la descarga de las bases de datos de paquetes, asegurándote de que no haya corrupción en los archivos locales).

2. Actualizar el sistema (El paso crítico)

Ahora intenta correr la actualización completa. Si te pregunta por reemplazar paquetes o conflictos, lee con atención:

Bash

```
pacman -Syu
```

⚠️ Qué hacer si `pacman` te da el error de "GLIBC_2.42 not found"

Si al intentar ejecutar el comando anterior `pacman` falla con el error de la librería (porque el propio binario de pacman está intentando usar la versión nueva que falta), tenemos que usar el **"Plan de Inyección"** que preparamos antes, pero ahora que sabemos que tus mirrors están bien:

1. **Sal del chroot:** `exit`
2. **Desde el Live USB, fuerza la instalación de las piezas rotas:**

Bash

```
sudo pacman -Syu glibc lib32-glibc pacman manjaro-system --root /mnt --overwrite "*"
```

3. Limpiar los Locales (Los warnings del principio)

Una vez que logres actualizar (ya sea dentro o fuera del chroot), vuelve a entrar al chroot y arregla los mensajes de "cannot change locale":

Bash

```
locale-gen
```

¿El comando `pacman -Syu` funcionó dentro del chroot o te bloqueó con el error de GLIBC? Si funcionó, ¡ya casi estás fuera de peligro! Solo deja que termine de descargar e instalar todo. Si te dio error, usa el comando del punto 2 (fuera del chroot).

> **You asked:**

> ✦ **Gemini Replied:**

Gemini ha dicho

Los archivos que subiste (trazas de `strace` de `locale-gen`) confirman exactamente lo que sospechábamos: el sistema está buscando los archivos de configuración de idioma (locales) en `/usr/lib/locale/es_AR.UTF-8/` y **no los encuentra** (`ENOENT`), por eso ves esos warnings de `setlocale`.

Sin embargo, **no pierdas tiempo arreglando los locales todavía**. El problema real es que tu base de datos de paquetes y las librerías del sistema están "desfasadas".

Ahora que ya tienes `pacman-mirrors` funcionando y apuntando a Uruguay, sigue estos pasos para terminar de reparar el sistema:

1. Sincronizar y Actualizar (Dentro del chroot)

Ejecuta esto para forzar a `pacman` a leer las listas nuevas que acabas de generar:

Bash

```
pacman -Syyu
```

2. Si `pacman` da error de "GLIBC_2.42 not found"

Es muy probable que `pacman` falle porque él mismo necesita la nueva `glibc`. Si eso pasa, **sal del chroot** (`exit`) y corre esto desde el Live USB:

Bash

```
sudo pacman -Syu glibc lib32-glibc manjaro-system pacman --root /mnt --overwrite "*"
```

Este comando es el "cirujano": usa el pacman sano del USB para instalar las piezas críticas en el disco duro (`/mnt`), pisando cualquier archivo corrupto.

3. Arreglar los Locales (Basado en tus archivos)

Una vez que la actualización del paso anterior termine (ya sea por el método 1 o 2), vuelve a entrar al chroot y ahora sí, genera los idiomas que faltan para que desaparezcan los errores que viste en tus archivos `strace`:

Bash

```
manjaro-chroot /mnt bash
# Editar el archivo de configuración (asegúrate de que es_AR.UTF-8 UTF-8 no tenga un '#' al princ
sed -i 's/#es_AR.UTF-8 UTF-8/es_AR.UTF-8 UTF-8/' /etc/locale.gen
# Generar
locale-gen
```

4. Reinstalar el Kernel (Por seguridad)

A veces, cuando `glibc` se rompe, el kernel no se actualiza bien. Para evitar que el sistema no arranque, corre esto antes de salir:

Bash

```
pacman -S $(pacman -Qsq "^linux[0-9]*$" | grep -v "utils")
```

Resumen de la situación según tus archivos: Tus logs muestran que el sistema intenta cargar `es_AR.UTF-8` pero las carpetas en `/usr/lib/locale/` están vacías o no existen. Esto es normal si la actualización se cortó. **Al correr `pacman -Syu`, esas carpetas se volverán a llenar correctamente.**

¿Lograste completar el `pacman -Syu` o te dio algún error de "conflictos de archivos"? Si hay conflictos, pásame las últimas líneas.

> **You asked:**

Has dicho

¿Sería posible conectar al display dentro de chroot?

qt.qpa.xcb: could not connect to display :0

> **Gemini Replied:**

Gemini ha dicho

Poder se puede, pero es un poco más complejo que un proceso normal porque el `chroot` es una "prisión" aislada que no tiene acceso por defecto a los sockets gráficos (X11 o Wayland) que se ejecutan en tu Live USB.

Para que una aplicación con interfaz (como algo basado en Qt/XCB) funcione dentro del chroot, tienes que "abrirle la puerta" desde afuera.

Aquí tienes los pasos para lograrlo:

1. Dar permiso en el sistema anfitrión (Live USB)

Antes de entrar al chroot, tienes que decirle a tu servidor gráfico que acepte conexiones de usuarios locales:

Bash

```
# Ejecuta esto en la terminal del Live USB (fuera del chroot)
xhost +local:
```

2. Montar carpetas temporales necesarias

Para que las aplicaciones vean el "enchufe" del monitor, el chroot necesita acceso a `/tmp`. Si usaste `manjaro-chroot`, esto suele hacerse solo, pero asegúrate de que el socket de X11 esté disponible:

Bash

```
# Verifica si ves esto desde AFUERA
ls /tmp/.X11-unix/
```

3. Pasar la variable DISPLAY al entrar

Al entrar al chroot, debes exportar la variable que le indica a las apps dónde está la pantalla. Generalmente es `:0`.

Bash

```
# Entra al chroot
manjaro-chroot /mnt bash

# Una vez adentro, exporta la variable
export DISPLAY=:0
export QT_X11_NO_MITSHM=1
```

4. ¿Por qué podrías seguir teniendo errores?

Si después de esto recibes el error `X11 connection rejected because of wrong authentication`, es porque te falta el archivo de "llaves" gráficas (`.Xauthority`).

Truco rápido para solucionar esto: Desde afuera (Live USB), copia tu llave al home del root del chroot:

Bash

```
sudo cp ~/.Xauthority /mnt/root/
```

⚠️ Una advertencia importante

Intentar ejecutar `sddm` o un entorno de escritorio completo **dentro de un chroot** no suele funcionar bien y puede causar bloqueos, porque esos programas intentan tomar control del hardware (GPU, teclado, ratón) que ya está siendo usado por el Live USB.

¿Qué programa específico estás intentando abrir? Si es para probar si la reparación de `glibc` funcionó, te recomiendo intentar con algo simple como `xterm` o `geany` antes de probar cosas pesadas como `sddm`.

Por cierto, si el comando `pacman -Syu` que lanzamos antes terminó bien, ¡tus problemas de librerías deberían estar resueltos! ¿Pudiste completar la actualización?

> **You asked:**

Has dicho

Tengo este problema.

Failed to import OpenPGP keys into temporary keyring: gpgme_engine_check_version() error: Invalid crypto engin

e

> **Gemini Replied:**

Gemini ha dicho

Este error es el "jefe final" de los problemas de librerías. El mensaje `Invalid crypto engine` ocurre porque `gpgme` (la librería que maneja las llaves de seguridad) está intentando usar un componente que no coincide con tu versión actual de `glibc` o está corrupto.

Como `pacman` depende de estas llaves para validar lo que descarga, estás en un callejón sin salida **dentro** del chroot. La única forma de romper este ciclo es usar el sistema exterior (el Live USB) para "reparar a la fuerza" el interior.

Sigue estos pasos estrictamente desde la terminal del **Live USB** (fuera del chroot):

1. Refrescar las llaves del Live USB

Primero asegúrate de que el sistema que va a hacer la reparación tenga las llaves al día:

Bash

```
sudo pacman -Sy archlinux-keyring manjaro-keyring
```

2. Reparación "Cirugía Mayor" (Sin entrar al chroot)

Vamos a reinstalar los paquetes base que están causando el error de GPG y GLIBC directamente en tu disco duro (montado en `/mnt`). Usaremos `--overwrite` para ignorar los archivos corruptos:

Bash

```
sudo pacman -Syu glibc lib32-glibc manjaro-system pacman gpgme --root /mnt --overwrite "*"
```

¿Qué pasa si este comando también te da error de llaves? Si incluso desde afuera te da error de GPG al intentar escribir en `/mnt`, usa este comando que **salta la verificación de llaves** solo por esta vez para poder rescatar el sistema:

Bash

```
sudo pacman -Syu glibc lib32-glibc manjaro-system pacman gpgme --root /mnt --overwrite "*" --gpgd
```

3. Reinstalar las llaves dentro del disco

Una vez que el paso anterior termine (esto reparará los binarios), el error de `Invalid crypto engine` debería desaparecer. Ahora ya puedes entrar al chroot y resetear las llaves de tu sistema instalado:

Bash

```
manjaro-chroot /mnt bash
# Ya adentro:
pacman-key --init
pacman-key --populate archlinux manjaro
pacman -Syu
```

¿Por qué llegamos a esto?

Tu sistema sufrió una "desincronización de base". Intentar ejecutar algo gráfico (`sddm`) o herramientas de sistema con la `glibc` rota es lo que disparó el error de GPGME. Es como intentar usar una llave moderna en una cerradura oxidada; el "motor criptográfico" simplemente no reconoce el entorno.

¿El comando del paso 2 logró terminar o te mostró una lista de "archivos en conflicto"? Si terminó, ya puedes intentar el paso 3 y tu sistema volverá a la vida.

> **You asked:**

Has dicho

¿Cómo desactivo GPG en DNF?

[main]

gpgcheck=0

localpkg_gpgcheck=0

skiptoken=1

Gemini Replied:

Gemini ha dicho

El error que estás viendo (**`Invalid crypto engine`**) confirma que el sistema de seguridad de paquetes está roto debido a la incompatibilidad de `glibc`. Intentar desactivar la verificación de firmas es una buena técnica de rescate para poder instalar las librerías que repararán el motor criptográfico.

Sin embargo, **Manjaro usa `pacman` (no DNF)**. Aunque la lógica es la misma, los archivos y comandos son distintos.

Aquí tienes cómo desactivar la verificación de firmas en Manjaro para forzar la reparación:

1. Desactivar GPG en `pacman.conf`

Debes editar el archivo de configuración de pacman **dentro de tu disco duro**.

1. Desde el Live USB, abre el archivo:

   Bash

   ```
   sudo nano /mnt/etc/pacman.conf
   ```

2. Busca la sección `[options]` y cambia (o añade) las siguientes líneas:

   Ini, TOML

   ```
   SigLevel = Never
   LocalFileSigLevel = Never
   ```

3. (Opcional) Si ves secciones para repositorios específicos como `[core]`, `[extra]`, etc., asegúrate de que también tengan `SigLevel = Never` o que estén comentadas para que hereden

la opción global.

4. Guarda con `Ctrl+O` y sal con `Ctrl+X`.

2. Ejecutar la reparación sin firmas

Ahora que pacman no preguntará por "motores criptográficos" ni llaves, intenta la inyección desde el Live USB:

Bash

```
sudo pacman -Syu glibc lib32-glibc manjaro-system pacman --root /mnt --overwrite "*"
```

3. ¿Por qué te interesa esto?

Al poner `SigLevel = Never`, pacman dejará de llamar a `gpgme` (el "crypto engine" que falla). Una vez que instales la nueva versión de `glibc` y `pacman` con el comando anterior, el error de `Invalid crypto engine` debería desaparecer porque las librerías volverán a ser compatibles entre sí.

4. ¡IMPORTANTE! Reestablecer la seguridad

Una vez que el sistema vuelva a arrancar y el error de GLIBC haya desaparecido:

1. Vuelve a dejar `SigLevel = Required DatabaseOptional` en `/etc/pacman.conf`.
2. Reinstala y actualiza las llaves correctamente:

   Bash

   ```
   sudo pacman-key --init
   sudo pacman-key --populate archlinux manjaro
   ```

Nota sobre DNF: Si estás intentando usar comandos de `dnf` porque viste tutoriales de Fedora, no funcionarán en Manjaro. Manjaro es un derivado de Arch Linux y su herramienta es siempre `pacman`.

¿El comando del paso 2 logró instalar los paquetes ahora que desactivaste las firmas? Si te da error de "database not found", corre primero `sudo pacman -Sy --root /mnt`.

> **You asked:**

Has dicho

Tengo este problema.

==> ERROR: module not found: 'radeon,usbhid'

Gemini Replied:

Gemini ha dicho

Ese error suele aparecer cuando intentas regenerar la imagen del kernel (`mkinitcpio`) y el archivo de configuración tiene un error de sintaxis. El sistema está buscando un módulo que literalmente se llama `"radeon,usbhid"` (con la coma incluida), y obviamente no existe.

En Linux, los módulos en la línea `MODULES` deben ir separados por **espacios**, no por comas.

Cómo solucionarlo

1. Corregir el archivo de configuración

Dentro de tu sistema (en el chroot), debes editar el archivo donde se definen los módulos del arranque:

Bash

```
nano /etc/mkinitcpio.conf
```

Busca la línea que empieza por `MODULES=` . Probablemente la tengas así: `MODULES=(radeon,usbhid)` o `MODULES="radeon,usbhid"`

Cámbiala por esto (separado por espacio):

Bash

```
MODULES=(radeon usbhid)
```

2. Regenerar el Kernel

Una vez guardado el archivo (`Ctrl+O` , `Enter` , `Ctrl+X`), ejecuta el comando para aplicar los cambios. Como estás reparando el sistema, lo mejor es hacerlo para todos los kernels instalados:

Bash

```
mkinitcpio -P
```

¿Por qué pasó esto?

Es muy probable que durante tus intentos de arreglar el problema gráfico (`sddm`) o los periféricos, editaras esta línea manualmente.

- **radeon**: Es el driver para tarjetas de video AMD antiguas.
- **usbhid**: Es el driver para teclados y ratones USB.

Si los pones con coma, el sistema se confunde y no carga ninguno de los dos, lo que podría dejarte sin teclado o sin imagen al arrancar.

Siguiente paso recomendado

Ahora que corregiste el error del módulo, ¿el comando `mkinitcpio -P` terminó sin errores?

Si terminó bien, intenta reinstalar el kernel por si acaso algo quedó a medias durante el problema de la `glibc` :

Bash

```
pacman -S linux
```

(O la versión que uses, como `linux-lts` o `linux-zen`).

¿Te falta algún otro error por resolver antes de intentar reiniciar? Estás muy cerca de tener el sistema estable de nuevo.

> **You asked:**

Has dicho

¿Por qué ocurre esto con dnf?

Failed to import OpenPGP keys into temporary keyring: gpgme_engine_check_version() error: Invalid crypto engin

e

> **Gemini Replied:**

Gemini ha dicho

El error `Invalid crypto engine` es una señal de que el "corazón" del sistema de seguridad de paquetes está roto. Aunque el mensaje mencione **DNF** (que es el gestor de Fedora), en el contexto de tu reparación en **Manjaro**, el problema es el mismo: la librería `gpgme` ha quedado inutilizable.

Aquí te explico la razón técnica y cómo saltarte este bloqueo para poder reparar el sistema.

¿Por qué ocurre?

La causa principal es la **incompatibilidad de versiones de la GLIBC**.

1. **Dependencia Rota:** El comando (ya sea `dnf` o `pacman`) llama a una librería llamada `gpgme` para verificar que los paquetes sean auténticos.
2. **Conflicto de Librerías:** `gpgme` necesita funciones específicas de la librería central del sistema (`glibc`). Como tu actualización quedó a medias, `gpgme` está intentando usar una versión de `glibc` que no encuentra o que tiene símbolos (funciones) diferentes.
3. **Fallo del "Motor":** Al no poder cargarse correctamente, `gpgme` informa que su "motor criptográfico" no es válido. Es como si el software intentara arrancar un coche, pero el motor no reconoce la gasolina que le cargaste.

Cómo solucionar el bloqueo de llaves

Dado que no puedes importar llaves porque el motor está roto, debes forzar la actualización **saltándote la verificación de firmas** solo una vez para poder instalar la `glibc` correcta que arreglará todo.

1. Si estás usando DNF (Fedora/RHEL):

Debes editar el archivo de configuración para desactivar el chequeo temporalmente:

Bash

```
# Editar la configuración
nano /etc/dnf/dnf.conf

# Añadir o cambiar estas líneas:
gpgcheck=0
localpkg_gpgcheck=0
```

Luego intenta la actualización nuevamente.

2. Si estás en Manjaro (donde usas pacman):

No uses comandos de DNF. Para desactivar la protección en Manjaro y arreglar el `crypto engine`:

1. **Edita el archivo:** `nano /etc/pacman.conf`
2. **Cambia la seguridad:** Busca `SigLevel` y ponlo en `Never`.
3. **Repara desde AFUERA (Live USB):**

   Bash

   ```
   sudo pacman -Syu glibc lib32-glibc manjaro-system pacman --root /mnt --overwrite "*"
   ```

El paso final tras la reparación

Una vez que logres actualizar y el sistema vuelva a reconocer la `glibc`, el error de `Invalid crypto engine` desaparecerá por sí solo porque las piezas volverán a encajar.

IMPORTANTE: En cuanto el sistema funcione, **vuelve a activar la seguridad** (cambia `Never` por `Required DatabaseOptional` en Manjaro, o `gpgcheck=1` en DNF) para que tu sistema no sea vulnerable a paquetes maliciosos.

¿Lograste editar el archivo de configuración para saltar este error?

> **You asked:**

Has dicho

¿Cómo cambio en dnf los https por http?

> **Gemini Replied:**

Gemini ha dicho

Para cambiar las URLs de **https** a **http** en los repositorios (ya sea para evitar problemas con certificados SSL o por compatibilidad con el error de "crypto engine"), debes editar los archivos `.repo` que se encuentran en `/etc/yum.repos.d/`.

Como estás en un entorno de recuperación (chroot), hazlo de la siguiente manera:

1. Ubicación de los archivos

Los archivos de configuración de DNF están en: `/etc/yum.repos.d/`

2. Cambio masivo usando `sed` (Recomendado)

En lugar de editar uno por uno, puedes usar el comando `sed` para buscar todas las instancias de `https` y cambiarlas a `http` en todos los archivos de repositorio a la vez.

Ejecuta esto (dentro o fuera del chroot apuntando a la ruta correcta):

Bash

```
# Si estás DENTRO del chroot:
sudo sed -i 's/https:/http:/g' /etc/yum.repos.d/*.repo

# Si estás AFUERA del chroot (Live USB):
sudo sed -i 's/https:/http:/g' /mnt/etc/yum.repos.d/*.repo
```

3. Cambio manual (Si prefieres verificar)

Si quieres hacerlo solo en el repositorio principal:

1. Abre el archivo (ejemplo para Fedora): `nano /etc/yum.repos.d/fedora.repo`
2. Busca las líneas que empiezan con `baseurl=` o `metalink=`.
3. Cambia el `https://...` por `http://...`.
4. Guarda con `Ctrl+O` y sal con `Ctrl+X`.

⚠️ Importante: ¿Por qué dnf sigue fallando?

Si estás haciendo esto para solucionar el error de **"Invalid crypto engine"**, ten en cuenta que cambiar de HTTPS a HTTP ayuda si el problema es de certificados (SSL), pero **no arregla el motor de firmas (GPG)**.

Para que DNF funcione ignorando por completo la seguridad (que es lo que necesitas ahora para rescatar el sistema), debes combinar el cambio a HTTP con la desactivación de GPG que mencionamos antes:

En tu `/etc/dnf/dnf.conf` o en los archivos `.repo`:

Ini, TOML

```
gpgcheck=0
repo_gpgcheck=0
```

¿Por qué estás usando DNF en Manjaro?

Solo una nota de seguridad: Si tu sistema original es **Manjaro**, los comandos de `dnf` no funcionarán aunque logres conectarte a los servidores, ya que Manjaro usa archivos `.pkg.tar.zst` (gestionados por `pacman`) y Fedora usa `.rpm` (gestionados por `dnf`).

Si tu sistema es Fedora, el comando `sed` anterior es la solución perfecta. Si es **Manjaro**, asegúrate de estar editando `/etc/pacman.d/mirrorlist` y cambiando las URLs allí.

¿Qué sistema operativo estás intentando recuperar exactamente? (Para darte las URLs de los espejos correctos si es necesario).